```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from google.colab import files
from IPython.display import display, clear_output
```

```
uploaded = files.upload()

video_name = list(uploaded.keys())[0]

print("Uploaded video:", video_name)
```

```
Choose Files 9208692-u…0_25fps.mp4
9208692-uhd_4096_2160_25fps.mp4(video/mp4) - 49088730 bytes, last modified: 9/12/2026 - 100% done
Saving 9208692-uhd_4096_2160_25fps.mp4 to 9208692-uhd_4096_2160_25fps.mp4
Uploaded video: 9208692-uhd_4096_2160_25fps.mp4
```

```
cap = cv2.VideoCapture(video_name)

if not cap.isOpened():
    print("Error: Could not open video.")
else:
    fps = cap.get(cv2.CAP_PROP_FPS)
    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    print("Video opened successfully")
    print("FPS:", fps)
    print("Width:", width)
    print("Height:", height)
    print("Total Frames:", total_frames)
```

```
Video opened successfully
FPS: 25.0
Width: 4096
Height: 2160
Total Frames: 679
```

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

if not ret or not ret2:
    print("Could not read frames.")
else:
    print("Two consecutive frames loaded successfully.")

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

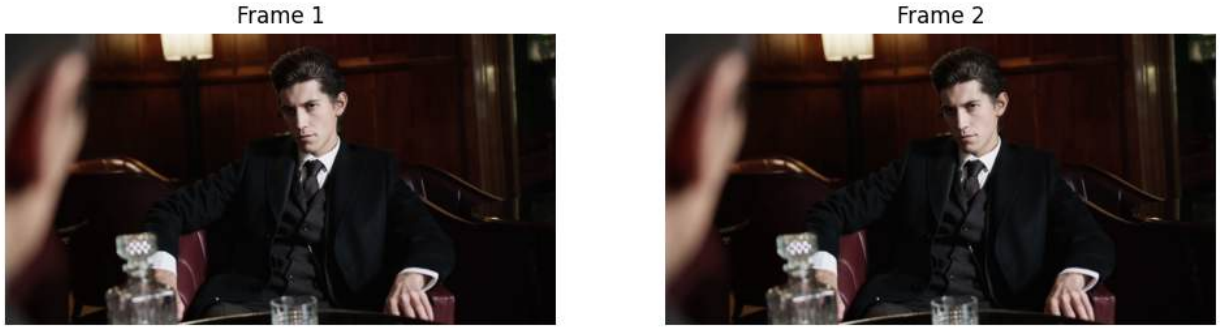
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(frame1, cv2.COLOR_BGR2RGB))
plt.title("Frame 1")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Frame 2")
plt.axis("off")

plt.show()
```

```
Two consecutive frames loaded successfully.
```




```
feature_params = dict(
    maxCorners=100,
    qualityLevel=0.3,
    minDistance=7,
    blockSize=7
)

p0 = cv2.goodFeaturesToTrack(
    gray1,
    mask=None,
    **feature_params
)

if p0 is None:
    print("No feature points detected.")
else:
    print("Number of Shi-Tomasi feature points:", len(p0))
```

```
Number of Shi-Tomasi feature points: 10
```

```
shi_tomasi_result = frame1.copy()

if p0 is not None:
    for point in p0:
        x, y = point.ravel()

        cv2.circle(
            shi_tomasi_result,
            (int(x), int(y)),
            5,
            (0, 255, 0),
            -1
        )

plt.figure(figsize=(10, 6))
plt.imshow(cv2.cvtColor(shi_tomasi_result, cv2.COLOR_BGR2RGB))
plt.title("Shi-Tomasi Detected Feature Points")
plt.axis("off")
plt.show()
```

Shi-Tomasi Detected Feature Points



```
lk_params = dict(
    winSize=(15, 15),
    maxLevel=2,
    criteria=(
        cv2.TERM_CRITERIA_EPS |
        cv2.TERM_CRITERIA_COUNT,
        10,
        0.03
    )
)

p1, st, err = cv2.calcOpticalFlowPyrLK(
    gray1,
    gray2,
    p0,
    None,
    **lk_params
)

if p1 is None:
    print("Optical flow could not be calculated.")
else:
    good_new = p1[st == 1]
    good_old = p0[st == 1]

    print("Tracked feature points:", len(good_new))
```

```
Tracked feature points: 10
```

```
lk_result = frame2.copy()

for new, old in zip(good_new, good_old):

    x_new, y_new = new.ravel()
    x_old, y_old = old.ravel()

    # Draw motion arrow
    cv2.arrowedLine(
        lk_result,
        (int(x_old), int(y_old)),
        (int(x_new), int(y_new)),
        (0, 255, 0),
        2,
        tipLength=0.3
```

```
    )

    # Mark new position
    cv2.circle(
        lk_result,
        (int(x_new), int(y_new)),
        4,
        (0, 0, 255),
        -1
    )

plt.figure(figsize=(12, 7))
plt.imshow(cv2.cvtColor(lk_result, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Motion Trajectories and Displacement Vectors")
plt.axis("off")
plt.show()
```




```
dx = good_new[:, 0] - good_old[:, 0]
dy = good_new[:, 1] - good_old[:, 1]

magnitude = np.sqrt(dx**2 + dy**2)

direction = np.degrees(np.arctan2(dy, dx))

# Convert negative angles to 0-360 degrees
direction = (direction + 360) % 360

print("Average displacement:", np.mean(magnitude), "pixels")
print("Maximum displacement:", np.max(magnitude), "pixels")
print("Average direction:", np.mean(direction), "degrees")
```

```
Average displacement: 3.5812569 pixels
Maximum displacement: 4.12016 pixels
Average direction: 13.434918 degrees
```

```
total_displacement = np.sum(magnitude)

print("Total displacement across tracked points:",
      total_displacement, "pixels")
```

```
Total displacement across tracked points: 35.81257 pixels
```

```
cap = cv2.VideoCapture(video_name)

ret, old_frame = cap.read()

if not ret:
    print("Error reading video.")
else:
    old_gray = cv2.cvtColor(old_frame, cv2.COLOR_BGR2GRAY)

    p0 = cv2.goodFeaturesToTrack(
        old_gray,
        mask=None,
        **feature_params
    )

    trajectory_mask = np.zeros_like(old_frame)

    frame_count = 0

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        if p0 is None:
            break

        p1, st, err = cv2.calcOpticalFlowPyrLK(
            old_gray,
            frame_gray,
            p0,
            None,
            **lk_params
        )

        if p1 is None:
            break

        good_new = p1[st == 1]
        good_old = p0[st == 1]

        for new, old in zip(good_new, good_old):

            x_new, y_new = new.ravel()
            x_old, y_old = old.ravel()

            cv2.line(
                trajectory_mask,
                (int(x_old), int(y_old)),
                (int(x_new), int(y_new)),
                (0, 255, 0),
                2
            )

            cv2.circle(
                frame,
                (int(x_new), int(y_new)),
                3,
                (0, 0, 255),
                -1
            )

        output = cv2.add(frame, trajectory_mask)
```

```
        # Display selected frames
        if frame_count % 20 == 0:
            clear_output(wait=True)

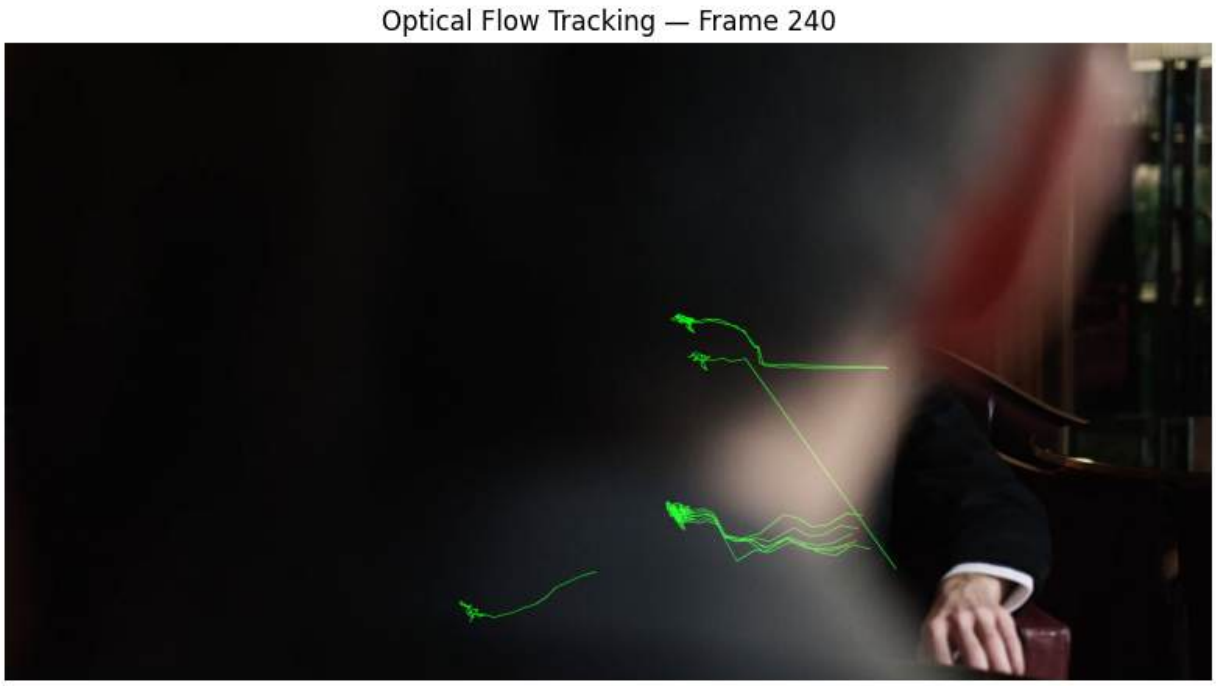
            plt.figure(figsize=(10, 6))
            plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
            plt.title(f"Optical Flow Tracking — Frame {frame_count}")
            plt.axis("off")
            plt.show()

        old_gray = frame_gray.copy()
        p0 = good_new.reshape(-1, 1, 2)

        frame_count += 1

    cap.release()

print("Tracking completed.")
```




```
Tracking completed.
```

```
cap = cv2.VideoCapture(video_name)

ret, frame1 = cap.read()
ret2, frame2 = cap.read()

cap.release()

gray1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(frame2, cv2.COLOR_BGR2GRAY)

flow = cv2.calcOpticalFlowFarneback(
    gray1,
    gray2,
    None,
    pyr_scale=0.5,
    levels=3,
    winsize=15,
    iterations=3,
    poly_n=5,
    poly_sigma=1.2,
    flags=0
)

print("Farneback Dense Optical Flow calculated successfully.")
```

```
Farneback Dense Optical Flow calculated successfully.
```

```
magnitude, angle = cv2.cartToPolar(
    flow[..., 0],
    flow[..., 1]
)

hsv = np.zeros_like(frame1)

hsv[..., 0] = angle * 180 / np.pi / 2
hsv[..., 1] = 255

hsv[..., 2] = cv2.normalize(
    magnitude,
    None,
    0,
    255,
    cv2.NORM_MINMAX
)

farneback_result = cv2.cvtColor(
    hsv,
    cv2.COLOR_HSV2BGR
)

plt.figure(figsize=(12, 7))
plt.imshow(
    cv2.cvtColor(
        farneback_result,
        cv2.COLOR_BGR2RGB
    )
)

plt.title("Farneback Dense Optical Flow")
plt.axis("off")
plt.show()
```




```
plt.figure(figsize=(18, 6))
```

```
plt.subplot(1, 3, 1)
plt.imshow(cv2.cvtColor(frame2, cv2.COLOR_BGR2RGB))
plt.title("Original Frame")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(cv2.cvtColor(lk_result, cv2.COLOR_BGR2RGB))
plt.title("Lucas-Kanade Sparse Flow")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(cv2.cvtColor(farneback_result, cv2.COLOR_BGR2RGB))
plt.title("Farneback Dense Flow")
plt.axis("off")

plt.show()
```




```
if fps > 0:
    average_speed = np.mean(magnitude) * fps

    print(
        "Approximate average speed:",
        average_speed,
        "pixels/second"
    )
else:
    print("FPS information unavailable.")
```

```
Approximate average speed: 10.706098 pixels/second
```

```
plt.figure(figsize=(12, 7))

plt.imshow(
    cv2.cvtColor(lk_result, cv2.COLOR_BGR2RGB)
)

plt.title(
    "Final Optical Flow Based Object Tracking Result"
)

plt.axis("off")
plt.show()

print("===== FINAL RESULTS =====")
print("Tracked feature points:", len(good_new))
print("Average displacement:", np.mean(magnitude), "pixels")
print("Maximum displacement:", np.max(magnitude), "pixels")
print("Average direction:", np.mean(direction), "degrees")
print("Approximate average speed:",
      np.mean(magnitude) * fps,
      "pixels/second")
```

Final Optical Flow Based Object Tracking Result



```
===== FINAL RESULTS =====
Tracked feature points: 0
Average displacement: 0.4282439 pixels
Maximum displacement: 37.02776 pixels
Average direction: 13.434918 degrees
Approximate average speed: 10.706098 pixels/second
```

Ques 1. How does object tracking differ from object detection?

Ans. Object detection identifies and locates an object in an image or video frame, while object tracking follows the movement of an already identified object across consecutive frames. Detection answers **"Where is the object?"**, whereas tracking answers **"Where is the object moving?"** Tracking therefore maintains the object's position and trajectory over time.

Ques 2. Explain how Optical Flow can be used for real-time object tracking.

Ans. Optical Flow estimates the movement of pixels between consecutive video frames. Feature points of a moving object can first be selected and then tracked using the Lucas-Kanade Optical Flow algorithm. The movement of these points is used to determine the object's displacement, direction, and trajectory. By continuously processing successive frames, the object's motion can be tracked in real time.

Ques 3. What is the role of Shi-Tomasi Corner Detection in the Lucas-Kanade Optical Flow algorithm?

Ans. Shi-Tomasi Corner Detection is used to identify strong and reliable feature points in an image, such as corners. These points are suitable for tracking because their positions can be detected more accurately between consecutive frames. Lucas-Kanade Optical Flow then tracks these selected feature points to estimate the object's movement.

Ques 4. Why is Optical Flow suitable for motion analysis in videos?

Ans. Optical Flow is suitable because it directly estimates the apparent motion of objects between consecutive frames. It provides information about the **direction, magnitude, displacement, and trajectory** of movement. This makes it useful for continuously analyzing motion patterns in video sequences without depending only on frame-by-frame object detection.

Ques 5. What challenges arise while tracking fast-moving or partially occluded objects?

Ans. Fast-moving objects may move a large distance between consecutive frames, making it difficult to correctly match their feature points. Partial occlusion can hide some of the object's features, causing tracked points to disappear or become inaccurate. Changes in illumination and camera movement can also reduce tracking accuracy.

Ques 6. Compare Optical Flow-based tracking with deep learning-based object tracking methods.

Ans. Optical Flow-based tracking estimates motion from changes between consecutive frames and can provide efficient motion information without requiring a large trained neural network. Deep learning-based tracking methods use trained models to recognize and track objects and can generally handle complex object appearances and different object categories more effectively. Optical Flow is often computationally simpler, while deep learning methods usually require greater computational resources and training data.

Ques 7. How can Optical Flow be used in traffic monitoring and autonomous driving systems?

Ans. Optical Flow can analyze the movement of vehicles, pedestrians, and other objects in video frames. In traffic monitoring, it can estimate vehicle direction, speed, displacement, and movement patterns. In autonomous driving, optical flow can help detect moving objects, understand the surrounding scene, and estimate motion of nearby vehicles and pedestrians.

Ques 8. Explain the effect of camera motion on Optical Flow estimation.

Ans. Camera movement causes many pixels in the scene to appear to move even when the objects themselves are stationary. Therefore, the optical flow contains both **camera-induced motion** and **actual object motion**. This makes it difficult to distinguish moving objects from a moving background and can reduce tracking accuracy. The experiment specifically considers camera movement as a factor affecting tracking performance.

Ques 9. Mention five real-world applications where motion analysis using Optical Flow is commonly employed.

Ans. Five real-world applications are:

1. Traffic monitoring
2. Autonomous navigation
3. Video surveillance
4. Human activity recognition
5. Sports analytics

Optical Flow is also useful for real-time object tracking and robotics.

Ques 10. How can Optical Flow improve the performance of surveillance, robotics, and human activity recognition systems?

Ans. Optical Flow provides continuous information about how objects and people move in a video. In surveillance, it can help detect and track unusual or important movements. In robotics, it can provide information about object and camera motion for navigation and interaction. In human activity recognition, motion patterns and trajectories can be analyzed to identify different activities and gestures. Thus, Optical Flow improves the ability of intelligent vision systems to understand dynamic scenes.